МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №2

по дисциплине «Объектно-ориентированное программирование»

Студент гр. 5345 _____ Пантелеев А. С.

Преподаватель _____ Шестопалов Р. П.

Санкт-Петербург

2026

СОДЕРЖАНИЕ

Цель работы.....3
1 Описание архитектуры.....3
2 Описание классов.....9
2.1 Класс Ability.....9
2.2 Класс Splash.....9
2.3 Класс RangedAttach.....9
2.4 Класс Heal.....9
2.5 Класс Teleport.....10
2.6 Класс Status.....10
2.7 Классы Slowdown и Overload.....10
2.8 Классы Burning и Shield.....10
2.9 Классы ItemPickup и AttackBoost.....11
2.10 Классы Cell и StatusConfig.....11
2.11 Класс Robot.....11
2.12 Класс Player.....11
2.13 Классы World и GameEvent.....12
2.14 Классы Factory и RobotConfig.....12
2.15 Классы AbilityMenu и Input.....12
2.16 Класс Game.....13
2.17 Классы Renderer и RenderCell.....13
2.18 Классы Point и PathFinder.....13
3 Тестирование.....14
Заключение.....20

Цель работы

Цель работы — расширить пошаговую игру с роботами: добавить способности и их улучшение, статусы роботов и клеток, предметы и режимы подсветки поля. При разработке нужно использовать наследование и полиморфизм, а также сохранить разделение игровой логики, ввода и вывода.

1 Описание архитектуры

Во второй лабораторной работе к игре добавлены четыре способности: удар вокруг, дальний удар, лечение и телепортация. У роботов появились статусы, у игрока — инвентарь. При повышении уровня можно получить новую способность или улучшить уже имеющуюся. Для работы с этими возможностями добавлены меню выбора, выбор цели на поле и цветовая подсветка.

Код разделён на группы abilities, statuses, units, world, navigation и menu в каталоге game. В terminal находятся классы ввода и отрисовки. Заголовки размещены в include, реализации — в src. Классы способностей и статусов работают с игровыми объектами, а выводом в консоль по-прежнему занимаются Renderer и Terminal.

Общий интерфейс Ability задаёт применение способности, расход энергии, радиус действия, тип и улучшение. Его реализуют Splash, RangedAttach, Heal и Teleport. Robot хранит набор объектов через std::unique_ptr<Ability>. Поэтому одному роботу можно выдать несколько способностей, а способ применения и параметры каждой из них определяются её классом.

Для способностей используется паттерн «Стратегия»: разные действия имеют общий интерфейс и выбираются во время выполнения программы. Game создаёт нужную способность по значению AbilityType в методе createAbility. Это простая фабрика: создание объекта и настройка его

начальных параметров собраны в одном месте. Игровой класс Factory отвечает за появление роботов на поле.

Вызов способности проходит через World. Сначала проверяется перегрузка, затем вызывается use нужной способности. Конкретная способность проверяет энергию и условия применения. Дальний удар и телепортация получают выбранную точку, лечение и удар вокруг выполняются без отдельной цели. При успешном применении робот расходует энергию; до завершения хода можно вызвать способность ещё раз.

Статусы представлены базовым классом Status и отдельными классами эффектов. Общие методы позволяют проверить окончание эффекта, объединить повторное наложение, заблокировать движение или применение способности, поглотить урон и обработать конец хода. Каждый класс хранит только нужное ему состояние: число ходов, оставшийся запас щита или признак срабатывания.

В Robot добавлены списки действующих и ожидающих статусов. В конце хода сначала обрабатываются текущие эффекты, затем удаляются завершившиеся и добавляются ожидающие. Такой порядок позволяет применять эффекты клеток и способностей с последующего хода. Повторный статус объединяется с уже имеющимся через repeat, поэтому правила продления или усиления находятся в самом классе эффекта.

Клетка хранит описания эффектов в StatusConfig. При создании мира на проходимой клетке появляется от нуля до двух эффектов; вероятности составляют 85, 12 и 3 процента. При входе робота World создаёт нужные статусы и передаёт их роботу. После срабатывания с клетки удаляется один случайный обычный статус. Предметы получает только игрок, после чего запись о подобранном предмете удаляется.

Инвентарь находится в Player и хранит количество предметов трёх типов. Предметы здоровья и энергии восстанавливают по 30 единиц, а предмет атаки добавляет AttackBoost на три хода. Для получения предмета

используется ItemPickup. Это позволяет размещать предметы на поле через тот же механизм описания эффектов клетки.

При повышении уровня World помещает GameEvent в очередь. Game извлекает события и открывает AbilityMenu. Если выбранной способности ещё нет, создаётся новый объект; если она уже есть, вызывается upgrade. Очередь отделяет начисление опыта от работы с клавиатурой: World сообщает о повышении уровня, а Game выбирает момент показа меню.

В код ЛР1 внесены изменения, необходимые для новых правил. Robot теперь учитывает щит при получении урона и прибавку к атаке от статусов. Player возвращает число полученных уровней из gainExp, чтобы для каждого повышения создать событие. Взаимодействие роботов возвращает InteractResult, а World обрабатывает уничтожение и выдачу опыта одинаково для ближней атаки и способностей.

Factory вместо готового образца Robot хранит RobotConfig. Роботы владеют способностями и статусами через уникальные указатели, поэтому для нового врага создаётся отдельный объект с заданными характеристиками и собственной случайной способностью лечения или телепортации. Такой способ создания сохраняет независимое состояние каждого робота.

Renderer теперь формирует поле из RenderCell, где отдельно хранятся символ, цвет текста и фон. Это нужно для четырёх режимов отображения и подсветки цели. В Point добавлен расчёт расстояния, а в PathFinder — поиск точки в заданном радиусе для телепортации противника. Input дополнен командами применения способности, использования предметов и переключения режима. Игровые объекты не зависят от терминального вывода.

На рисунках 1–3 приведены UML-диаграммы классов ЛР2 и связанных с ними классов ЛР1. На рисунке 1 показаны способности и развитие роботов, на рисунке 2 — статусы и предметы, на рисунке 3 — управление и визуализация. На схемах сохранены связи между показанными классами. У классов предыдущей работы указаны новые и изменённые методы.

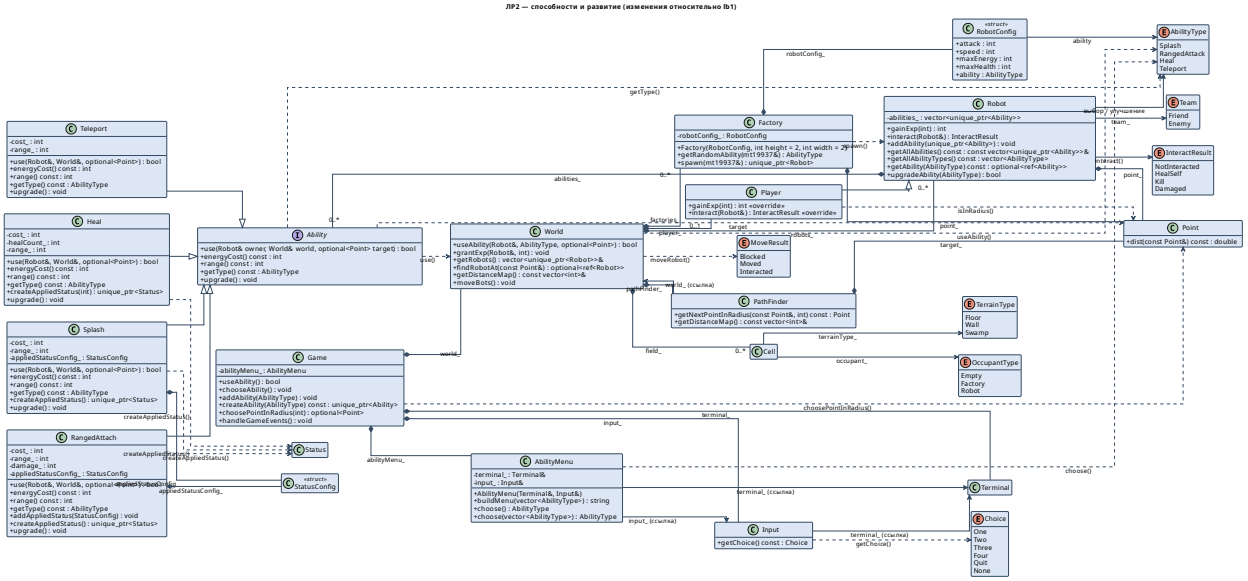


Рисунок 1 – UML-диаграмма способностей и развития роботов

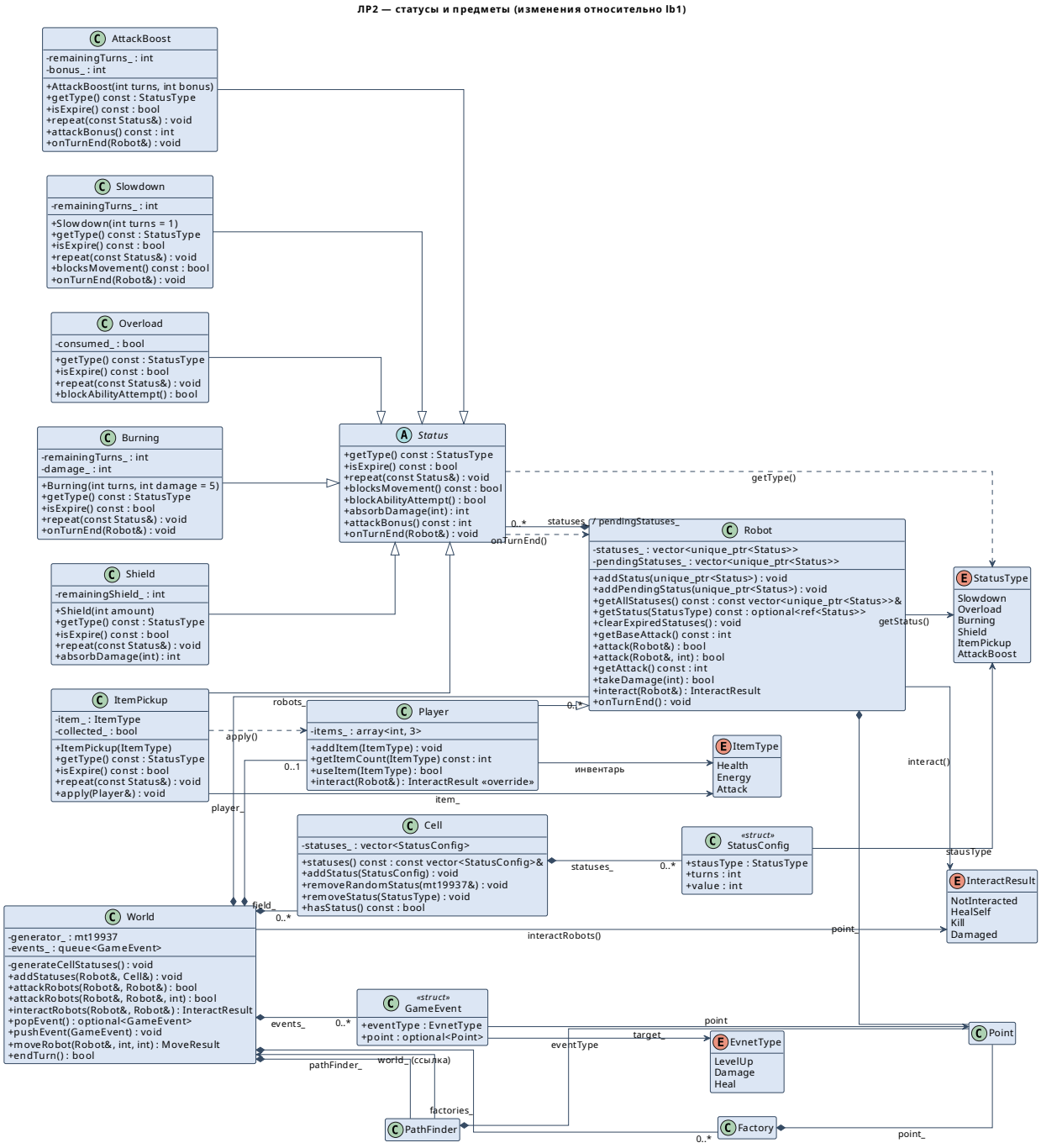


Рисунок 2 – UML-диаграмма статусов и предметов

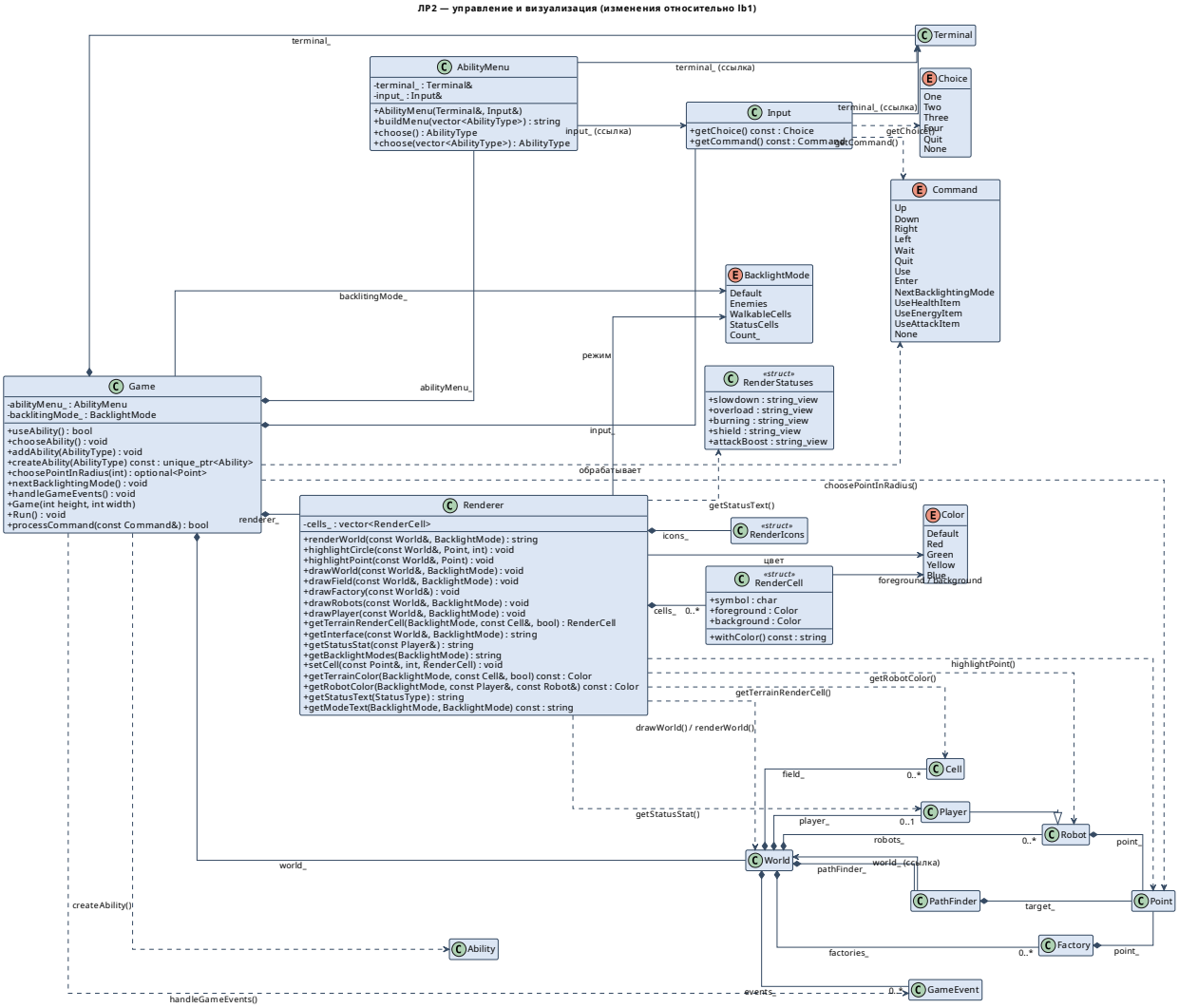


Рисунок 3 – UML-диаграмма управления и визуализации

2 Описание классов

2.1 Класс Ability

Ability задаёт общий интерфейс способности без хранения её состояния. Метод use выполняет действие, energyCost и range сообщают параметры, а getType позволяет найти способность в наборе робота. Метод upgrade нужен для улучшения существующего объекта без его замены. Виртуальный деструктор обеспечивает корректное удаление конкретной способности через базовый указатель.

2.2 Класс Splash

Splash наносит урон вражеским роботам вокруг владельца. При поиске целей учитываются расстояние и команда робота. Цели сначала собираются отдельно, чтобы уничтожение одного противника не мешало обработать остальных. Выжившим целям передаётся перегрузка. Метод upgrade увеличивает радиус удара на одну клетку.

2.3 Класс RangedAttach

RangedAttach реализует дальний удар по выбранному противнику. Перед атакой проверяются наличие цели, расстояние, команда и энергия владельца. Начальный урон равен 15, радиус — трём клеткам, расход энергии — пяти единицам. Выживший противник получает горение. Метод addAppliedStatus задаёт параметры этого эффекта, а upgrade увеличивает урон на пять.

2.4 Класс Heal

Heal восстанавливает владельцу 20 единиц здоровья за пять единиц энергии. Здоровье ограничивается максимальным значением. Если величина лечения превышает недостающее здоровье, избыток превращается в ожидающий статус Shield. Метод createAppliedStatus создаёт щит нужной величины. После upgrade объём лечения увеличивается на пять.

2.5 Класс Teleport

Teleport переносит робота на выбранную свободную и проходимую клетку в радиусе пяти клеток. Проверка занятости защищает от переноса в клетку другого робота или фабрики. Перемещение выполняется через World, поэтому позиция и состояние клеток обновляются вместе. Начальный расход энергии равен десяти; upgrade уменьшает его на единицу, сохраняя минимум в одну единицу.

2.6 Класс Status

Status объединяет работу с эффектами робота. Методы getType, isExpire и repeat определяют вид эффекта, момент его окончания и повторное наложение. Остальные виртуальные методы позволяют эффекту участвовать только в нужной операции: движении, применении способности, расчёте атаки, получении урона или конце хода. Например, щиту нужен absorbDamage, а горению — onTurnEnd.

2.7 Классы Slowdown и Overload

Slowdown хранит оставшееся число ходов и запрещает движение, пока это число положительно. Метод onTurnEnd уменьшает длительность, а repeat складывает длительности при повторном наложении.

Overload блокирует одну попытку применить способность. Метод blockAbilityAttempt отмечает срабатывание, после чего эффект считается завершившимся. Повторное наложение не увеличивает число блокируемых попыток.

2.8 Классы Burning и Shield

Burning хранит длительность и урон за ход. В onTurnEnd он передаёт урон владельцу и уменьшает число оставшихся ходов. Повторное наложение продлевает действие эффекта.

Shield хранит оставшийся запас поглощения. Метод absorbDamage уменьшает этот запас и возвращает только ту часть урона, которая должна попасть в здоровье. Повторное наложение увеличивает общий запас щита.

2.9 Классы ItemPickup и AttackBoost

ItemPickup хранит тип предмета и признак получения. Метод apply передаёт предмет игроку только один раз. World вызывает его именно для Player, поэтому обычный робот не забирает предмет с клетки.

AttackBoost хранит прибавку к атаке и число оставшихся ходов. Метод attackBonus возвращает прибавку, а onTurnEnd уменьшает длительность. Повторное наложение продлевает время действия. Основная атака робота сохраняется отдельно от временного бонуса.

2.10 Классы Cell и StatusConfig

В Cell добавлен набор StatusConfig. Структура хранит тип эффекта, число ходов и числовой параметр: урон, запас щита или тип предмета. Сам объект статуса создаётся при наступании робота на клетку. Методы addStatus и removeStatus меняют состав эффектов, removeRandomStatus удаляет один случайный обычный эффект после срабатывания, а hasStatus используется при подсветке поля.

2.11 Класс Robot

Robot хранит способности, действующие и ожидающие статусы. Методы поиска возвращают необязательную ссылку: вызывающий код может проверить наличие нужного объекта и использовать его без передачи владения. Повторное добавление одинаковой способности не создаёт копию, а улучшение выполняется отдельным методом upgradeAbility.

Методы attack и takeDamage отделяют нанесение урона от взаимодействия при движении. При расчёте атаки учитываются действующие бонусы, а входящий урон последовательно обрабатывается щитами. Метод onTurnEnd отвечает за эффекты, восстановление энергии и подключение ожидающих статусов.

2.12 Класс Player

Player дополнен инвентарём. Метод addItem увеличивает количество предметов, getItemCount нужен для интерфейса, а useItem проверяет

возможность использования и уменьшает количество только после успешного действия. При полном здоровье или энергии соответствующий предмет сохраняется.

Метод gainExp возвращает число полученных уровней. Это позволяет отдельно обработать выбор способности при каждом повышении. При росте уровня базовая атака увеличивается независимо от временного AttackBoost.

2.13 Классы World и GameEvent

World связывает способности и статусы с состоянием поля. Он применяет эффекты клетки, запускает способности, обрабатывает атаки и начисляет опыт. Метод findRobotAt возвращает необязательную ссылку, поскольку выбранная для дальнего удара клетка может оказаться пустой.

GameEvent содержит тип события и необязательную позицию. World складывает события повышения уровня в очередь, а Game получает их через popEvent. Методы pushEvent и grantExp позволяют сообщить о результате игрового действия без обращения к меню из World.

2.14 Классы Factory и RobotConfig

RobotConfig хранит начальные характеристики создаваемого робота и тип способности. Factory использует эти данные при spawn, создаёт отдельную способность и передаёт её новому роботу. Генератор случайных чисел передаётся в spawn, чтобы выбор способности выполнялся тем же генератором, который использует мир.

2.15 Классы AbilityMenu и Input

AbilityMenu выводит варианты и возвращает выбранный AbilityType. Один вариант choose предлагает все четыре способности, другой принимает список уже доступных роботу. Метод buildMenu строит текст для переданного списка и сохраняет соответствие номера конкретной способности.

Input переводит клавиши в Command или Choice. Разделение нужно потому, что цифры в обычной игре используют предметы, а внутри меню

выбирают способность. Клавиша E открывает применение способности, Tab меняет режим подсветки, Enter подтверждает выбранную точку.

2.16 Класс Game

Game управляет выбором и применением способностей. Метод createAbility создаёт конкретный объект, chooseAbility добавляет или улучшает его, а useAbility определяет, требуется ли выбор цели. Метод choosePointInRadius выводит поле с границей радиуса и курсором, обрабатывает движение курсора, подтверждение и отмену.

Метод handleGameEvents открывает меню после повышения уровня. В processCommand добавлены действия для способностей, предметов и подсветки. Переключение вида поля и работа с меню выполняются отдельно от завершения игрового хода.

2.17 Классы Renderer и RenderCell

RenderCell хранит символ клетки, цвет текста и фон. Метод withColor добавляет управляющие последовательности терминала. Renderer сначала заполняет массив таких клеток, затем собирает итоговую строку. Это позволяет отдельно менять внешний вид местности, роботов, границы радиуса и выбранной точки.

Методы getTerrainColor и getRobotColor задают подсветку для выбранного BacklightMode. В режиме WalkableCells выделяются непроходимые клетки. Методы highlightCircle и highlightPoint помогают выбрать цель способности, а getStatusStat выводит действующие эффекты игрока. Структура RenderStatuses объединяет цветные подписи статусов.

2.18 Классы Point и PathFinder

В Point добавлен dist для расчёта расстояния. Его используют видимость и радиусы способностей. Метод getNextPointInRadius класса PathFinder выбирает точку в радиусе телепортации по карте расстояний до игрока.

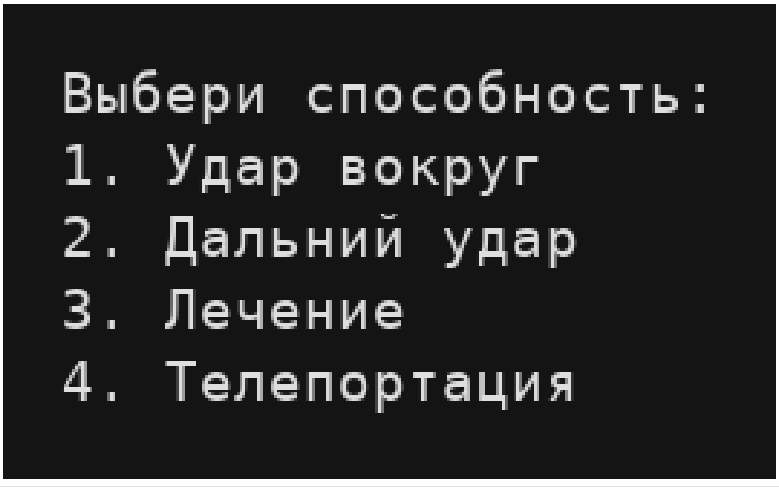
3 Тестирование

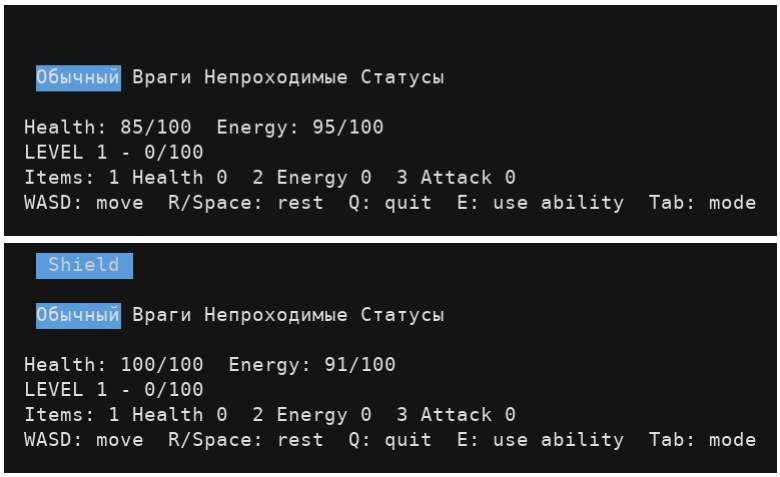
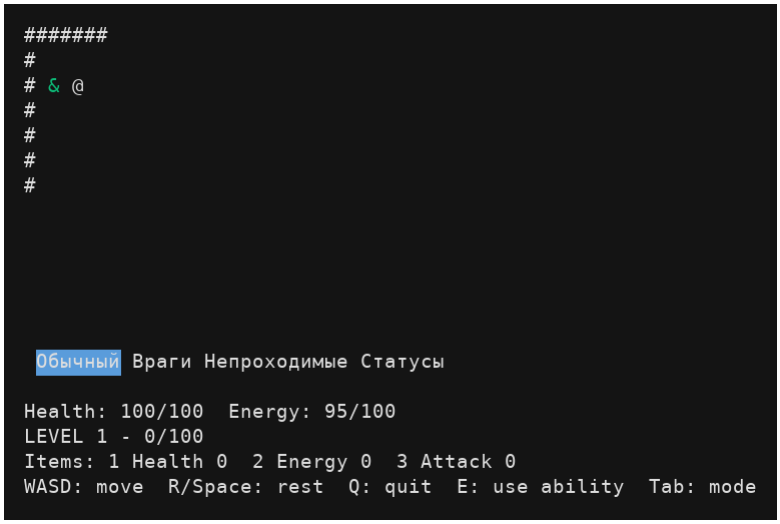
Проект собран через CMake с компилятором GCC и стандартом C++20. Я проверил игру в обычном запуске: начальный выбор способности, меню применения, четыре режима отображения, перемещение курсора цели, отмену выбора и выход. Для отдельных игровых ситуаций использована проверочная программа с теми же классами World, Robot, Player, способностей и статусов.

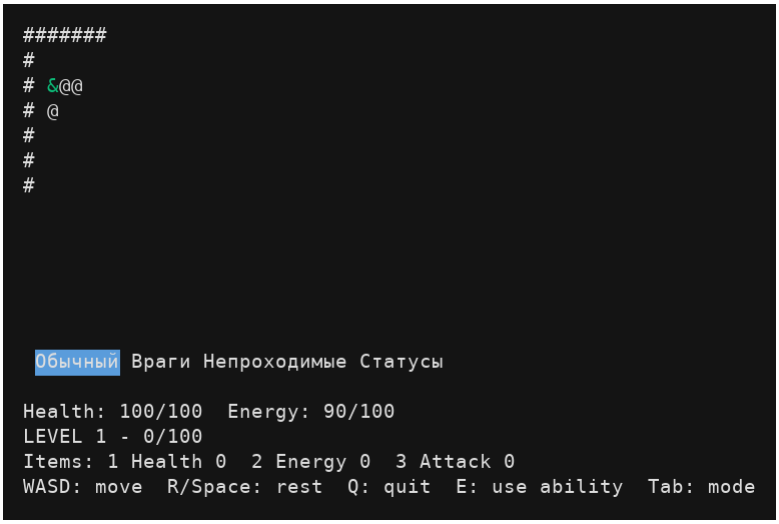
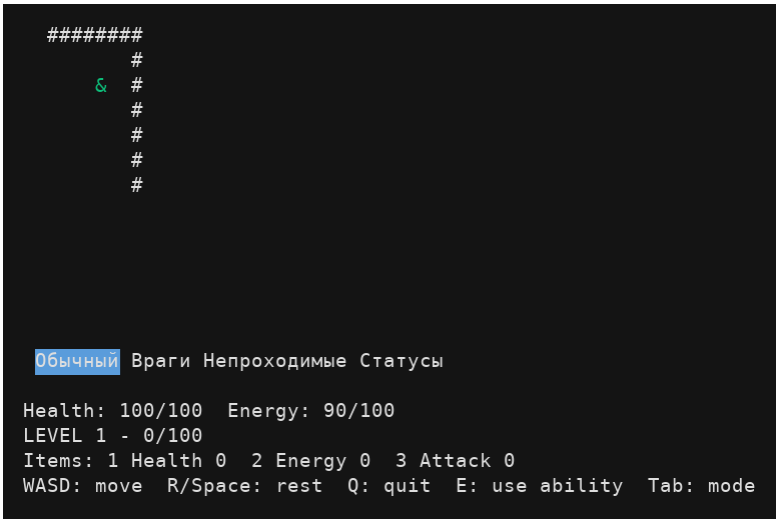
В проверочных сценариях задавались здоровье, энергия, позиции роботов и состав эффектов клеток. После действия проверялись новые значения и состояние объектов. Всего выполнено 47 проверок. Они охватывают четыре способности и их улучшения, ограничения применения, повторное наложение статусов, предметы, повышение уровня, очередь событий и создание робота фабрикой.

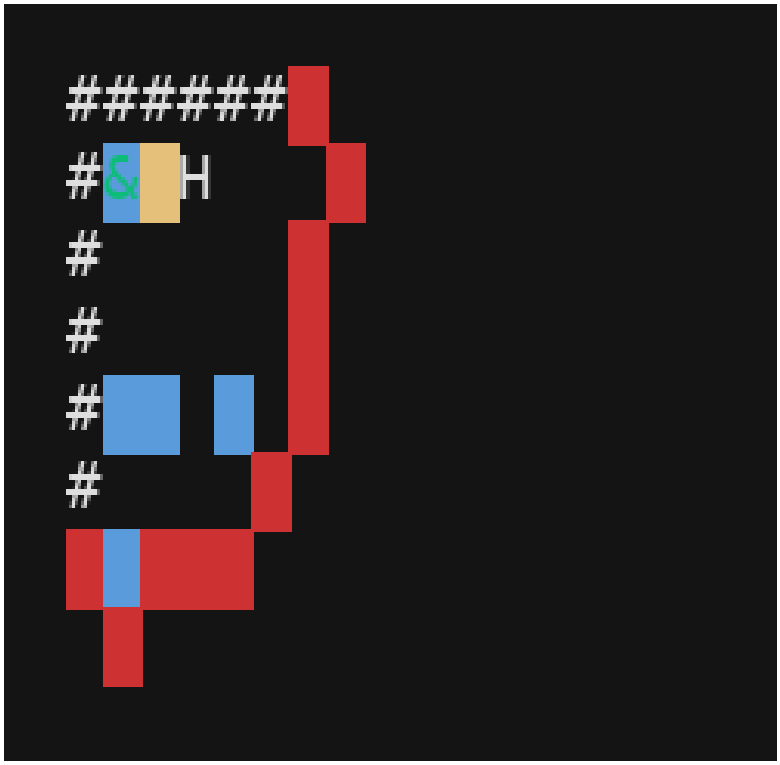
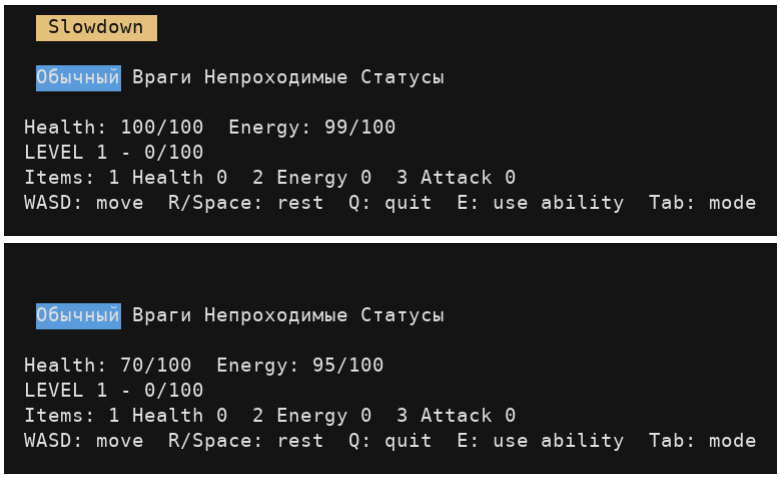
Снимки для таблицы 1 получены из обычного запуска и через штатный Renderer в проверочных сценариях. Проверялись не только успешные действия, но и отказ при нехватке энергии, пустой или недопустимой цели, а также использование предмета при полном здоровье и пустом инвентаре.

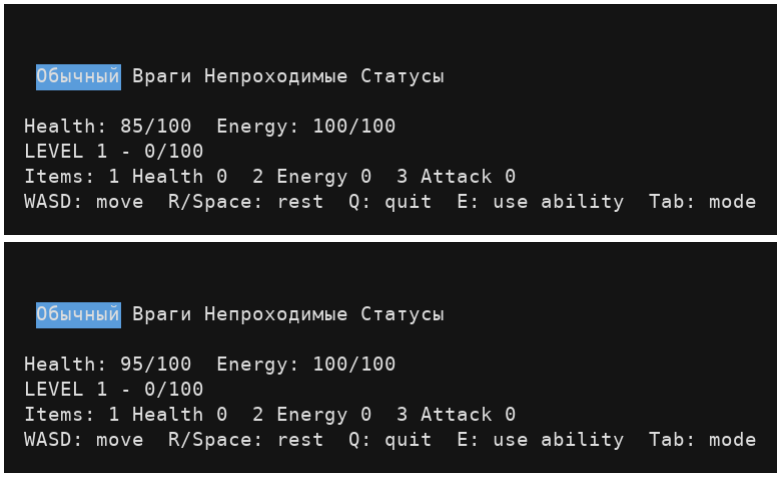
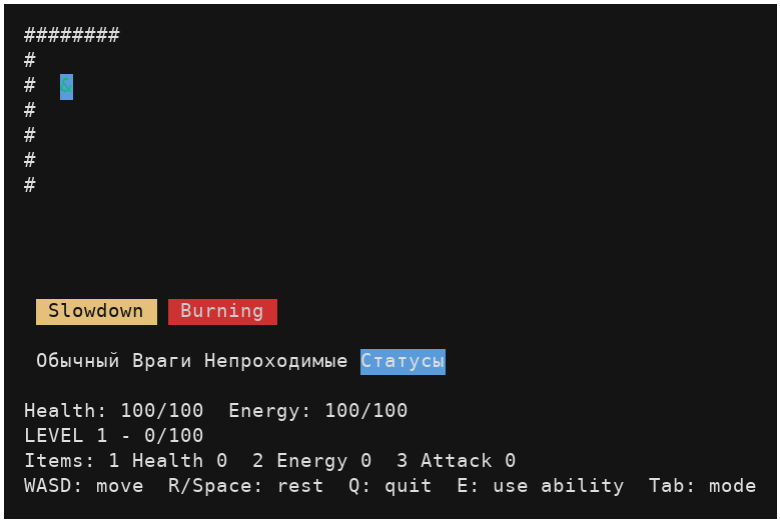
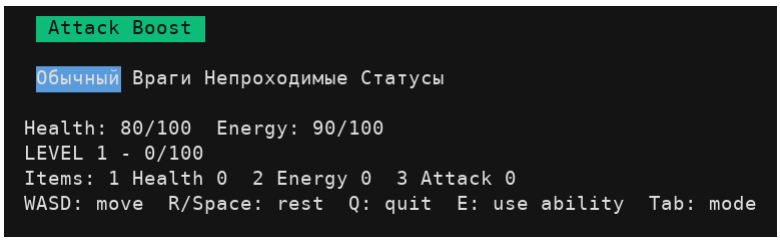
Таблица 1 – результаты проверки программы

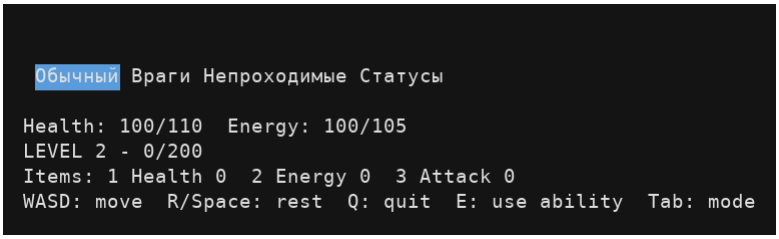
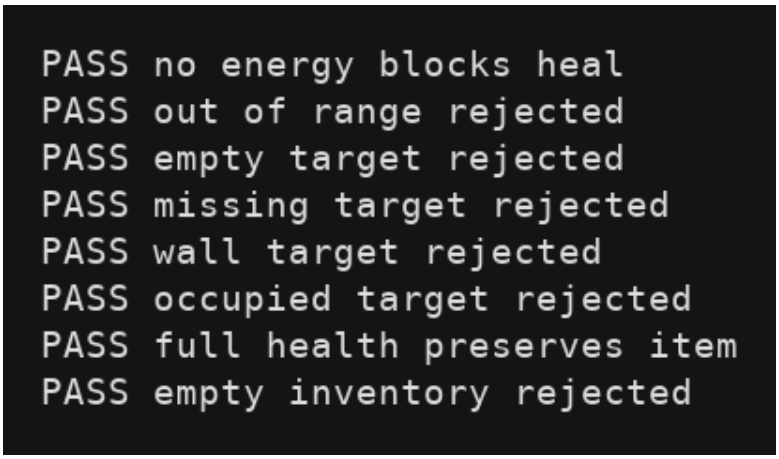
Входные данные	**Выходные данные**
Запуск игры и выбор способности	В начальном меню доступны четыре способности. Выбрана телепортация; игра перешла к отображению поля. 

Входные данные	Выходные данные
Лечение при здоровье 65 и энергии 100	Здоровье стало 85, энергия — 95. После улучшения лечение восстанавливает 25 единиц; избыток лечения создаёт щит. 
Дальний удар по противнику в радиусе двух клеток	Здоровье противника уменьшилось со 100 до 85, энергия игрока — до 95. Противнику добавлено горение. После улучшения урон вырос до 20. 

Входные данные	Выходные данные
Удар вокруг по двум соседним противникам	Оба противника получили по 10 единиц урона и перегрузку; робот на расстоянии двух клеток не пострадал. После улучшения он также попал в радиус. 
Телепортация из точки (2, 2) в (6, 2)	Позиция изменилась на (6, 2), энергия уменьшилась со 100 до 90. Исходная клетка освободилась. После улучшения стоимость стала равна 9. 

Входные данные	Выходные данные
Выбор цели в обычном запуске	После выбора телепортации показаны граница радиуса и выбранная клетка. Курсор перемещается клавишами WASD, Q отменяет выбор. 
Повторное замедление и перегрузка	Два замедления запретили движение на два хода. Две перегрузки заблокировали одну попытку лечения без расхода энергии; следующая попытка восстановила здоровье с 50 до 70. 

Входные данные	Выходные данные
Повторное горение и щит	Горение на два и на один ход нанесло суммарно 15 единиц урона за три хода. Щиты на 10 и 5 поглотили 15 единиц из удара на 20; здоровье стало 95. 
Вход на клетку с двумя статусами	На роботе появились замедление и горение. С клетки удалился один из двух обычных эффектов. В режиме статусов оставшийся эффект выделен цветом. 
Получение и использование предметов	Предмет с клетки получил только игрок. Здоровье выросло с 50 до 80, энергия — с 60 до 90. Предмет атаки добавил 5 на три хода; после этого бонус исчез. 

Входные данные	Выходные данные
Получение 100 единиц опыта	Уровень стал равен 2, в очереди появилось событие LevelUp. Проверены улучшение имеющегося лечения и добавление телепортации. 
Переключение режимов отображения	Проверены обычный режим, подсветка противников, непроходимых клеток и клеток со статусами. Слева показана подсветка противника, справа — клетки со статусом. Состояние мира не меняется. 
Недопустимые действия	Без энергии способность не применяется. Дальний удар отклоняет пустую и слишком далёкую цель. Телепортация не выполняется в стену или занятую клетку. При полном здоровье предмет лечения сохраняется. 

Заключение

В ходе лабораторной работы игра расширена способностями, статусами и предметами. Игрок может выбирать и улучшать способности, применять дальний удар, удар вокруг, лечение и телепортацию. Роботы и клетки поддерживают эффекты, а инвентарь позволяет восстанавливать здоровье и энергию и временно повышать атаку.

Для способностей и статусов использованы общие интерфейсы и полиморфные объекты. World обрабатывает игровые действия и события, Game управляет меню, а Renderer выводит поле с выбранной подсветкой. Сборка, 47 проверок классов и обычный запуск подтвердили работу добавленных возможностей.